Project Plan: Particles Bouncing and Sticking on a Floor

Javad Yousefian, Alexandra VonHaven, Kevin Geumhan

October 6, 2026

1 Description of the simulation

In this project we want to simulate small particles (for example sand grains or droplets) that are shot from a small source on the floor. Each particle flies under gravity and air drag and then lands on the floor. When it lands, it can stick to the floor or bounce again. We want to find where the particles finally stop on the floor and how many times they bounce. We chose this problem because it is simple, there is only one surface (the floor), and many results can be calculated by hand, so we can check our simulation easily.

1.1 Setup

The floor is the plane $z = 0$ and the gravity is in the $-z$ direction. All particles start at the origin. The particles do not interact with each other, so we can keep all of them in numpy arrays and move them together. We use units with $m = 1$ and $g = 1$. We plan to run with and without air drag.

1.2 Equation of motion

During the flight, each particle follows Newton's second law with gravity and linear air drag:

$$m\frac{d^2\vec{r}}{dt^2} = -mg\,\hat{z} - \gamma\frac{d\vec{r}}{dt} \tag{1}$$

where γ is the drag coefficient. We write it as two first order equations, $d\vec{r}/dt = \vec{v}$ and $d\vec{v}/dt = -g\hat{z} - (\gamma/m)\vec{v}$. We put all positions and velocities in one long array and we move all flying particles one time step with `scipy.integrate.solve_ivp`. For this equation the exact solution is known, so we can use it to test the solver. With $\gamma = 0$ it is just projectile motion.

1.3 Landing on the floor

After every time step we check which particles have $z < 0$. These particles landed during the step. We find the landing point with linear interpolation between the old and the new position. Then for each landed particle:

- With probability p_{stick} the particle sticks. We save its final position and the number of bounces, and it does not move anymore.
- Otherwise it bounces. With probability p_s the bounce is specular: we change the sign of v_z, like a ball on a hard floor. In the other case the bounce is diffuse: the particle keeps its speed but gets a new random direction.

All bounces are elastic (no energy is lost), so when there is no drag the energy of each particle stays the same. The simulation stops when all particles are stuck.

1.4 Probability distributions

We need random numbers for these things. Here u and u' are uniform random numbers between 0 and 1.

- Starting speeds: we use the Maxwell Boltzmann distribution, $f(v) \propto v^2 e^{-v^2/2\sigma^2}$, like particles that come from a hot source. It is not easy to invert the CDF of this distribution, so we use rejection sampling. We take random points uniformly in a rectangle around the curve and we keep the points that are under the curve.
- Starting directions: particles leave the source with the cosine law, $p(\theta) \propto \cos\theta \sin\theta$, where θ is the angle from the vertical. With the inverse CDF method this gives $\theta = \arcsin\sqrt{u}$ and $\phi = 2\pi u'$.
- Diffuse bounces: the new direction also follows the cosine law, so we use the same inverse CDF formula.
- Sticking and specular bounces: we compare a uniform random number with p_{stick} and with p_s.

1.5 Main result

The main results that we want from the simulation are:

- the distribution of the final positions of the particles on the floor, especially the distance r from the source,
- the distribution of the number of bounces before a particle sticks,
- the difference between specular and diffuse bounces, and between flights with and without drag. With specular bounces and no drag, a particle keeps going in the same direction, so it should go very far. With diffuse bounces the direction changes every time, so the particle should do a random walk and stay closer to the source.

1.6 How we will check the results

- Range without drag: a particle with speed v and angle θ from the vertical lands at distance $R = v^2 \sin(2\theta)/g$. We compare every first landing with this formula. We can also calculate the mean value for our distributions, $\langle R \rangle = 3\pi\sigma^2/4g$, and compare it with the simulation.
- Number of bounces: each landing is independent, so the number of bounces k should follow $P(k) = (1 - p_{\text{stick}})^k \, p_{\text{stick}}$, with mean $(1 - p_{\text{stick}})/p_{\text{stick}}$.
- Energy: without drag the energy $\frac{1}{2}mv^2 + mgz$ of every particle should not change, also after the bounces.
- Symmetry: the source is the same in all directions, so the mean x and mean y of the final positions should be zero, and the distribution of the angle ϕ of the final position should be uniform.
- Random walk: for diffuse bounces, the mean r^2 should grow linearly with the number of flights, because the directions of the flights are independent.

- Drag: with drag, a particle that falls for a long time should reach the terminal velocity $v_t = mg/\gamma$.
- Random samples: we compare the histograms of our samples with the distributions that they should follow, and we check the acceptance rate of the rejection sampling.

1.7 Errors, complexity and levels of detail

We plan to study four kinds of errors. For finite precision, we check how much the energy changes over many bounces without drag. For numerical integration, we compare `solve_ivp` with the exact solution with drag for different tolerances. For the time step, we compare the first landing distance with the formula $R = v^2 \sin(2\theta)/g$ for different Δt, because we only find the landing at the end of each step. For the statistical error, the error of the mean distance should become smaller like $1/\sqrt{N}$.

For the levels of detail we use $N = 1000$, 10000 and 100000 particles. The particles do not interact, so we expect that the time for one step grows linearly with N. We will measure the time of the code for different N and use `cProfile` to find the slowest parts. We think `solve_ivp` will be the slowest part.

2 Pseudocode

```
read input parameters (N, dt, gamma, p_stick, p_specular, seed)
make random number generator

floor = Floor(p_stick, p_specular)
particles = Particles(N)
    speeds:     Maxwell Boltzmann (rejection sampling)
    directions: cosine law (inverse CDF)
    positions:  all at the origin

while some particles are still flying:
    save old positions
    particles.move(dt)                # solve_ivp for flying particles

    landed = particles with z < 0
    find landing points (linear interpolation)
    floor.land(particles, landed)
        stick with probability p_stick: save final position
        else specular bounce with probability p_specular
        else diffuse bounce (new direction, same speed)

    count the bounces of every particle

plot the final positions and the distance r
plot the number of bounces
compare with theory
```

3 Code structure

We will make one Python package with the name `bounce`, and it can be installed with pip. In the `pyproject.toml` file we list numpy, scipy and matplotlib as dependencies. When we install the package, it also installs a command `bounce` with three options: `bounce run`, `bounce validate` and `bounce benchmark`. We plan to write these files:

- `sampling.py`: functions for random sampling, `maxwell_speeds()` (rejection sampling) and `cosine_directions()` (inverse CDF).
- `particles.py`: the `Particles` class. It keeps the positions, velocities, number of bounces and a flag for stuck particles in numpy arrays. Methods: `move(dt)` (one `solve_ivp` step for the flying particles), `energy()` and `n_flying()`.
- `floor.py`: the `Floor` class with p_{stick} and p_s. Methods: `find_landed()` and `land()`, which does the sticking and the bounces.
- `simulation.py`: the function `run_simulation()` with the main loop. It returns the final positions, the number of bounces and the first landing distances.
- `analysis.py`: theory formulas, the validation checks, the benchmark and the plots.
- `cli.py`: the `main()` function for the command line.

We will also write some tests with pytest. Maybe some parts will change when we start to write the code. We will use black for the formatting and we will write numpydoc docstrings for all functions and classes.